

Java classes: a quick tour

just enough Java to read the stacks & queues code

AGENDA

- what a class is (and what an object is)
- public vs. private
- references, null, and == vs. .equals
- a class that points to its own type
- private inner classes
- a little Java housekeeping

A CLASS

- A class is a blueprint: it bundles data (fields) with behavior (methods).

```
1 public class Book {  
2     private String title;  
3     private int pages;  
4  
5     public Book(String title, int pages) {  
6         this.title = title;  
7         this.pages = pages;  
8     }  
9  
10    public String getTitle() {  
11        return title;  
12    }  
13 }
```

fields: each Book object gets its own copy

constructor: runs when you say new Book(...)

method: something a Book can do

SAME IDEA, IN PYTHON

```
1 class Book:
2     def __init__(self, title, pages):
3         self.title = title
4         self.pages = pages
5
6     def get_title(self):
7         return self.title
```

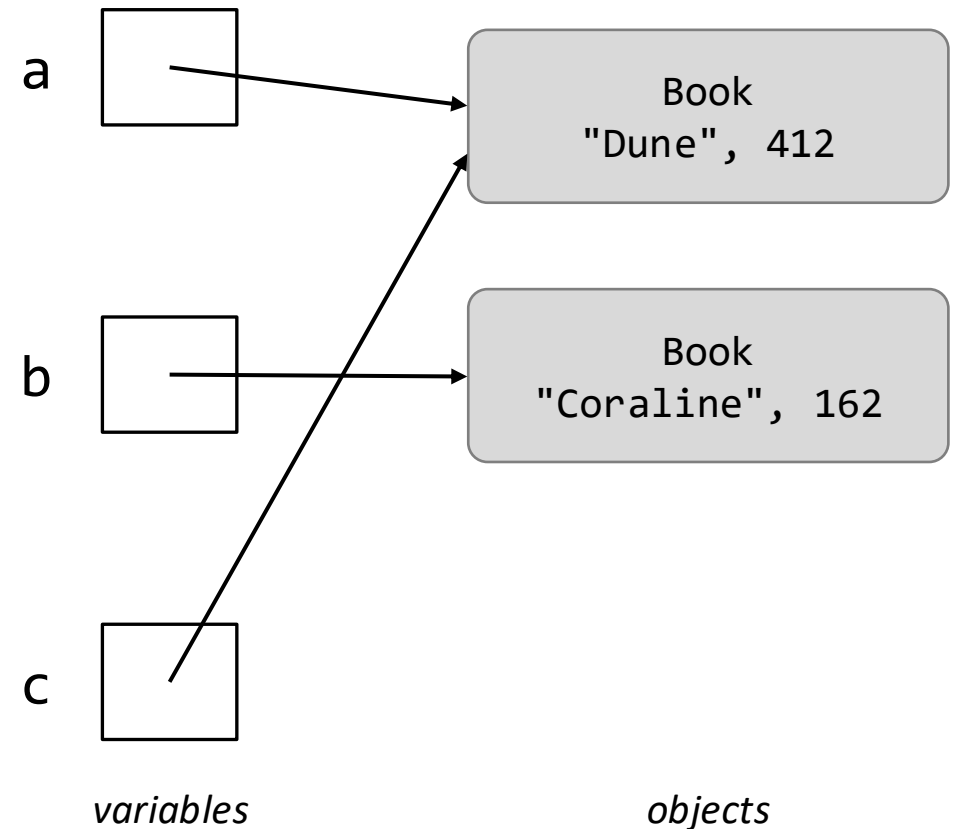
- Java declares its fields, with types, before using them.
- Java's this is Python's self, and you can usually leave it off.
- The constructor is named after the class, not `__init__`.
- Braces and semicolons instead of colons and indentation.

OBJECTS AND REFERENCES

- new builds an object and hands back a reference to it.

```
1 Book a = new Book("Dune", 412);  
2 Book b = new Book("Coraline", 162);  
3 System.out.println(a.getTitle());  
4  
5 Book c = a;
```

no copy! c and a now refer to the same Book
(so c == a is true)



PUBLIC VS. PRIVATE

- public: any code, anywhere, can use it.
- private: only code inside this class can touch it.

Book.java

```
1 public class Book {  
2     private String title;  
3     private int pages;  
4  
5     public String getTitle() { ... }  
6     public boolean isLong() { ... }  
7 }
```

BookDemo.java (a different file: outside Book)

```
1 Book a = new Book("Dune", 412);  
2  
3 a.getTitle();           // fine: public  
4 a.title = "Oops";      // nope: private
```



the locked door is really locked out
here

THE COMPILER SAYS NO

- Uncomment that line and Eclipse complains before you ever hit Run:

```
BookDemo.java:14: error: title has private access in Book
    a.title = "Oops";
      ^
```

1 error

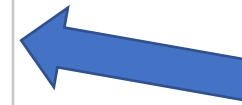
- This is not a crash. Nothing ran at all.
- Java checked the rules while building the program and refused to build it.
- A whole category of mistakes gets caught here instead of in front of a user.

WHY BOTHER MAKING FIELDS PRIVATE?

- Because then the class gets to enforce its own rules.

in Book.java

```
1 public void setPages(int pages) {  
2     if (pages < 1) {  
3         System.out.println("Books need a page!");  
4         return;  
5     }  
6     this.pages = pages;  
7 }
```



the rule lives in exactly one
place: inside Book

in BookDemo.java

```
1 a.setPages(-5);    // refused, politely
```

- If pages were public, anyone could write `a.pages = -5` and Book could do nothing about it.

NULL AND .EQUALS

- null means "refers to no object" (like Python's None).

```
1 Book d = null;  
2 d.getTitle();    // NullPointerException!
```

- For strings, == asks "same object?" not "same text?"

```
1 String s1 = "Dune";  
2 String s2 = new String("Dune");  
3  
4 s1 == s2      // false  
5 s1.equals(s2) // true
```

use .equals to compare strings. Every time.



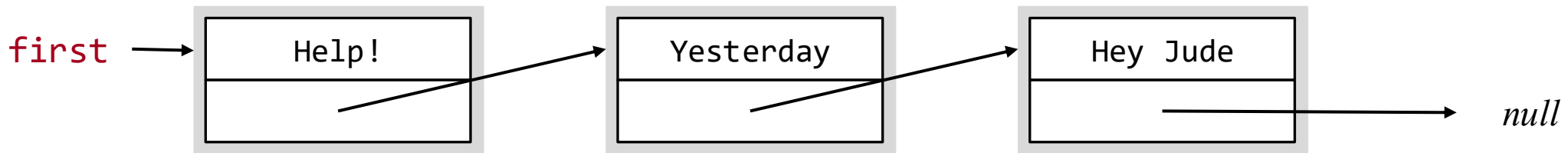
A CLASS THAT POINTS TO ITS OWN TYPE

```
1 class Node {  
2     String song;  
3     Node next;  
4 }
```

a reference to another Node (or null)



- Each Node holds one song, plus a link to the next Node.
- Chain them together and you get a linked list.



PRIVATE INNER CLASSES

```
1 public class Playlist {  
2     private class Node {  
3         String song;  
4         Node next;  
5     }  
6  
7     private Node first = null;  
8  
9     public void addToFront(String song) {  
10        Node newNode = new Node();  
11        newNode.song = song;  
12        newNode.next = first;  
13        first = newNode;  
14    }  
15 }
```



- Node lives inside Playlist.
- private: only Playlist can see it or use it.
- Code that uses a Playlist just calls addToFront. It never knows Node exists.

A LITTLE JAVA HOUSEKEEPING

- One public class per file, and the file is named after it: Book.java
- Programs start in a main method, which lives inside a class:
 - `public static void main(String[] args)`
- Every variable and field has a declared type, checked before the program runs.
- Braces mark blocks; semicolons end statements. Indentation is for humans.
- No need to free memory: the garbage collector reclaims objects nothing refers to.

WHERE THIS IS GOING

- Next up: stacks and queues.
- StackOfStrings will be a public class.
- Its Node will be a private inner class, the exact Playlist pattern.
- And the linked-list stack's push is... addToFront.